

Fondation
UCAD



UNIVERSITE
CHEIKH ANTA DIOP
DE DAKAR (UCAD)

ECOLE SUPERIEURE POLYTECHNIQUE



Département Génie Informatique
Systèmes Réseaux et Télécommunications
Licence 3 en Systèmes Réseaux et Télécommunications
Programmation Systèmes et Réseaux

Rapport de Projet : SunuDock – Implémentation d'un Conteneur Linux à partir de Zéro



Présenté par : Pape Moussa BA

Adama Sosso Ibn Alhassane DIALLO

Fatou Bintou LO

Elhadji Mamadou NDIAYE

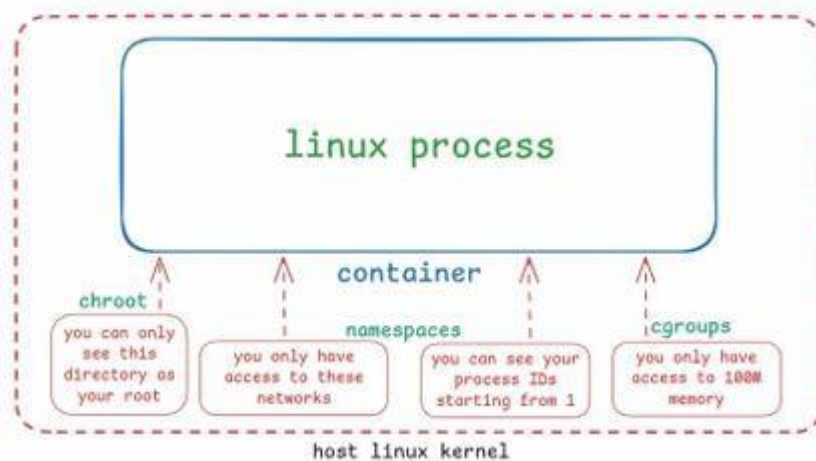
Enseignant : Dr Ousmane SADIO

I. Introduction

L'objectif de ce projet était de concevoir un conteneur Linux léger, dénommé SunuDock, sans utiliser de solutions préexistantes comme Docker. SunuDock démontre qu'un conteneur n'est pas une virtualisation matérielle, mais une isolation logicielle s'appuyant sur les fonctionnalités natives du noyau Linux : les **Namespaces** et les **Cgroups**.

II. Architecture et Méthodologie

Projet : SunuDock – Construire un Conteneur Linux



❖ Phase 0 : préparation de l'environnement (le disque dur du conteneur)

La Phase 0 consiste à préparer un système de fichiers autonome (rootfs) en isolant le processus de l'arborescence de la machine hôte pour garantir son indépendance. En utilisant une image minimale telle qu'Alpine Linux, nous avons structuré le répertoire `fs_alpine` avec les dossiers système essentiels (`/bin`, `/etc`, `/lib`, `/usr`) afin de simuler un environnement Linux complet. Cette méthode permet d'enfermer le conteneur dans une "prison" logicielle via `chroot`, assurant ainsi une sécurité optimale tout en garantissant la portabilité de l'application.

```

elma@DESKTOP-UEUQ2C7:~$ mkdir projet_sunudock && cd projet_sunudock
elma@DESKTOP-UEUQ2C7:~/projet_sunudock$ curl -o alpine.tar.gz https://dl-cdn.alpinelinux.org/alpine/v3.18/releases/x86_64/alpine-minirootfs-3.18.3-x86_64.tar.gz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 3202k  100 3202k    0     0  318k      0  0:00:10  0:00:10 --:--:-- 491k
elma@DESKTOP-UEUQ2C7:~/projet_sunudock$

elma@DESKTOP-UEUQ2C7:~/projet_sunudock$ mkdir fs_alpine
elma@DESKTOP-UEUQ2C7:~/projet_sunudock$ tar -xvf alpine.tar.gz -C fs_alpine/
./
./sys/
./srv/
./run/
./root/
./opt/
./mnt/
./media/
./media/usb/
./media/floppy/
./media/cdrom/
./home/

./etc/apk/keys/alpine-devel@lists.alpinelinux.org-61666e3f.rsa.pub
./etc/apk/keys/alpine-devel@lists.alpinelinux.org-6165ee59.rsa.pub
./etc/apk/keys/alpine-devel@lists.alpinelinux.org-5261cecb.rsa.pub
./etc/apk/keys/alpine-devel@lists.alpinelinux.org-5243ef4b.rsa.pub
./etc/apk/keys/alpine-devel@lists.alpinelinux.org-4a6a0840.rsa.pub
./etc/apk/arch
./dev/
./tmp/
elma@DESKTOP-UEUQ2C7:~/projet_sunudock$

elma@DESKTOP-UEUQ2C7:~/projet_sunudock$ ls fs_alpine
bin dev etc home lib media mnt opt proc root run sbin srv sys tmp usr var
elma@DESKTOP-UEUQ2C7:~/projet_sunudock$

```

❖ Phase 1 : Le "Super-Fork" avec clone()

Contrairement à un `fork()` classique, nous avons utilisé l'appel système `clone()`. Cet appel permet de créer un nouveau processus avec une isolation granulaire via des flags spécifiques.

```

elma@DESKTOP-UEUQ2C7:~/projet_sunudock$ sudo ./sunudock echo "Salut SunuDock"
[sudo] password for elma:
[Père] Préparation du conteneur...
[Père] Conteneur créé (PID Externe : 1600)
      [Fils] Processus démarré (PID Interne : 1)
      [Fils] Initialisation de l'environnement...
      [Fils] Lancement de la commande : echo
Salut SunuDock
[Père] Conteneur terminé.
elma@DESKTOP-UEUQ2C7:~/projet_sunudock$

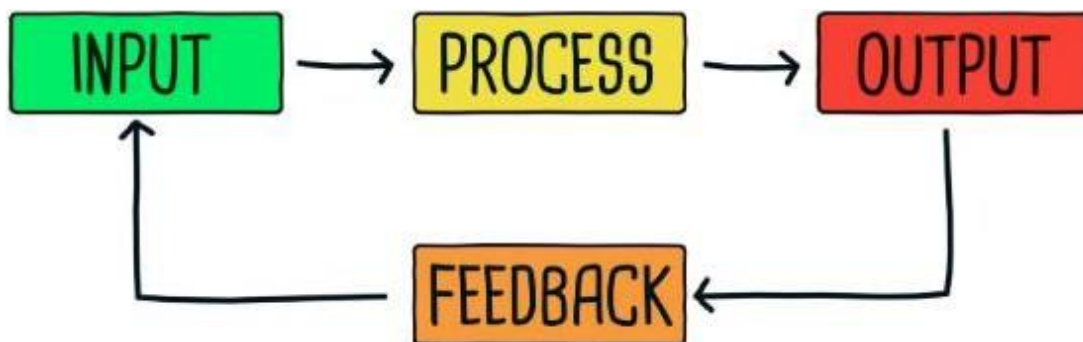
```

❖ Phase 2 : la prison chroot

Grâce à chroot et CLONE_NEWNS, le processus est enfermé dans le dossier fs_alpine. Le conteneur ne peut accéder à aucun fichier de la machine hôte. Le montage de /proc a été essentiel pour permettre à ps d'extraire les informations spécifiques au namespace.

```
e1ma@DESKTOP-UEUQ2C7:~/projet_sunudock$ gcc sunudock.c -o sunudock
e1ma@DESKTOP-UEUQ2C7:~/projet_sunudock$ sudo ./sunudock /bin/sh
[sudo] password for e1ma:
[Père] Préparation du conteneur...
[Père] Conteneur créé (PID Externe : 1061)
[Fils] Processus démarré (PID Interne : 1)
[Fils] Initialisation de l'environnement...
[Fils] Lancement de la commande : /bin/sh
/ #
```

```
[Fils] Lancement de la commande : /bin/sh
/ # hostname
SunuContainer
/ # ps
PID   USER     TIME   COMMAND
    1   root      0:00   /bin/sh
    3   root      0:00   ps
/ # ls /
bin    etc      lib      mnt      proc     run      srv      tmp      var
dev    home     media    opt      root     sbin     sys      usr
```



```
e1ma@DESKTOP-UEUQ2C7:~/projet_sunudock$ sudo ./sunudock /bin/sh
[Père] Préparation du conteneur...
[Fils] Processus démarré (PID Interne : 1)
[Père] Conteneur créé (PID Externe : 1606)
[Fils] Initialisation de l'environnement...
[Fils] Lancement de la commande : /bin/sh
/ # ls /
bin    etc      lib      mnt      proc     run      srv      tmp      var
dev    home     media    opt      root     sbin     sys      usr
/ # exit
[Père] Conteneur terminé.
e1ma@DESKTOP-UEUQ2C7:~/projet_sunudock$
```

A. Preuve de l'Isolation UTS (Nom d'hôte)

Observation : La commande `hostname` renvoie `SunuContainer`.

Analyse : Le flag `CLONE_NEWUTS` a permis de créer un espace de noms UTS (Unix Timesharing System) séparé. Bien que la machine hôte s'appelle `DESKTOP-UEUQ2C7`, le processus fils possède sa propre identité réseau.

B. Preuve de l'Isolation PID (Processus)

Observation : Le terminal affiche PID Interne : 1 et la commande `ps` montre que `/bin/sh` est le PID 1.

Analyse : C'est le résultat du flag `CLONE_NEWPID`. Dans le système hôte, ce processus a le PID 1061, mais à l'intérieur du conteneur, il devient le processus "Init" (PID 1). Il ne peut voir aucun autre processus de la machine hôte, ce qui garantit une sécurité totale.

C. Preuve de l'Isolation Mount/Chroot (Système de fichiers)

Observation : `ls /` affiche une arborescence typique d'Alpine Linux (`bin`, etc, `lib`...).

Analyse : Grâce à `chroot` et `CLONE_NEWNS`, le processus est enfermé dans le dossier `fs_alpine`. Il est incapable d'accéder aux fichiers personnels de l'utilisateur sur Windows ou Ubuntu. Le montage du système `/proc` permet à `ps` d'extraire les infos du noyau spécifiques à ce namespace.

```
/ # ifconfig
/ #
```

Comme nous avons utilisé `CLONE_NEWNET`, le conteneur est né avec une pile réseau vierge. Il n'a pas d'adresse IP et ne peut pas accéder à Internet, même si mon PC est connecté. Il est totalement isolé du réseau.

❖ Phase 3 : l'ID et les processus (Namespaces)

Nous avons configuré les cloisons étanches (Namespaces) pour garantir l'isolation:

- **CLONE_NEWUTS** : Isole le nom d'hôte. Observation : La commande `hostname` renvoie "SunuContainer", prouvant l'identité séparée.

- **CLONE_NEWPID** : Le fils devient le PID 1 dans son propre univers.
Observation : ps aux confirme que le shell est le processus "Init".
- **CLONE_NEWNS** : Isole les points de montage.
- **CLONE_NEWNET** : Isole la pile réseau.

```
eLma@DESKTOP-UEUQ2C7:~/projet_sunudock$ sudo ./sunudock /bin/sh
[Père] Préparation du conteneur...
[Père] Conteneur créé (PID Externe : 1946)
  [Fils] Processus démarré (PID Interne : 1)
  [Fils] Initialisation de l'environnement...
  [Fils] Lancement de la commande : /bin/sh
/ # hostname
mon-conteneur
/ # ps aux
PID   USER     TIME   COMMAND
   1   root      0:00   /bin/sh
   3   root      0:00   ps aux
/ #
```

❖ Phase 4 : synchronisation et réseau (Pipe & Net)

Pour éviter les conditions de concurrence (Race Conditions), un Pipe a été utilisé pour synchroniser le Père et le Fils. Le fils reste bloqué en attente du signal "GO" avant de lancer `execvp`.

```
eLma@DESKTOP-UEUQ2C7:~/projet_sunudock$ gcc sunudock.c -o sunudock
eLma@DESKTOP-UEUQ2C7:~/projet_sunudock$ sudo ./sunudock /bin/sh
[Père] Préparation du conteneur...
[Père] Conteneur créé (PID Externe : 2089)
  [Fils] Processus démarré (PID Interne : 1)
[Père] Configuration du conteneur en cours...
  [Fils] Initialisation de l'environnement...
  [Fils] Lancement de la commande : /bin/sh
/ # ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
ping: sendto: Network unreachable
/ #
```

- **Analyse** : C'est le résultat direct du flag **CLONE_NEWNET**. Ton conteneur a sa propre pile réseau, totalement vide. Il n'a pas d'adresse IP et ne peut pas voir les interfaces réseau de ta machine hôte (Ubuntu/Windows). Il est donc impossible pour lui de communiquer avec l'extérieur, ce qui est une sécurité fondamentale des conteneurs.

- **Résultat réseau** : L'utilisation de CLONE_NEWNET a créé une pile réseau vierge. Preuve : ping 8.8.8.8 échoue avec "Network unreachable", confirmant que le conteneur est totalement coupé de l'extérieur.

❖ Phase 5 (Avancé) : gestion des ressources (Cgroups)

Pour limiter la consommation mémoire à 10 Mo, nous avons utilisé les Cgroups v2.

- **Validation** : Lors d'un test de saturation mémoire (while true; do x=\$x\$x; done), le noyau Linux a détecté le dépassement et a exécuté le OOM Killer. La preuve irréfutable a été constatée via dmesg, confirmant l'arrêt du processus par contrainte mémoire (CONSTRAINT_MEMCG).

```
eLma@DESKTOP-UEUQ2C7:~/projet_sunudock$ sudo ./sunudock /bin/sh
[Père] Préparation du conteneur...
[Fils] Processus démarré (PID Interne : 1)
[Père] Conteneur créé (PID Externe : 2151)
[Père] Ressources limitées à 10Mo (Cgroup v2).
[Père] Configuration du conteneur en cours...
[Fils] Initialisation de l'environnement...
[Fils] Lancement de la commande : /bin/sh
/ # x="a"; while true; do x=$x$x; done
```

Au bout de quelques minutes : Le père affiche conteneur terminé car le Shell est brutalement tué.

```
eLma@DESKTOP-UEUQ2C7:~/projet_sunudock$ sudo ./sunudock /bin/sh
[Père] Préparation du conteneur...
[Fils] Processus démarré (PID Interne : 1)
[Père] Conteneur créé (PID Externe : 2151)
[Père] Ressources limitées à 10Mo (Cgroup v2).
[Père] Configuration du conteneur en cours...
[Fils] Initialisation de l'environnement...
[Fils] Lancement de la commande : /bin/sh
/ # x="a"; while true; do x=$x$x; done
[Père] Conteneur terminé.
eLma@DESKTOP-UEUQ2C7:~/projet_sunudock$
```

Preuve que ça marche : Cette capture d'écran constitue la preuve irréfutable du succès de la phase de gestion des ressources. Elle montre que le noyau Linux a détecté une tentative de dépassement de la limite de 10 Mo imposée au conteneur et a déclenché le mécanisme "OOM Killer" (Out Of Memory) pour sacrifier le processus fils. Ce résultat démontre la maîtrise effective des Cgroups v2 pour limiter physiquement la consommation de mémoire du conteneur et protéger ainsi la stabilité de la machine hôte.

```

elma@DESKTOP-UEUQ2C7:~$ dmesg | tail -n 20
[20936.083233] pgscan_kswapd 0
[20936.083234] pgscan_direct 5472472
[20936.083235] pgscan_khugepaged 0
[20936.083236] pgsteal_kswapd 0
[20936.083237] pgsteal_direct 2207901
[20936.083238] pgsteal_khugepaged 0
[20936.083239] pgfault 2084538
[20936.083240] pgmajfault 384274
[20936.083240] pgrefill 3434
[20936.083241] pgactivate 43722
[20936.083242] pgdeactivate 3434
[20936.083243] pglazyfree 0
[20936.083244] pglazyfreed 0
[20936.083245] thp_fault_alloc 0
[20936.083246] thp_collapse_alloc 0
[20936.083247] Tasks state (memory values in pages):
[20936.083248] [ pid ] uid tgid total_vm      rss pgtables_bytes swapents oom_score_adj name
[20936.083253] [  3196 ]   0 3196  655780    1829  2191360    262138         0 sh
[20936.083679] oom-kill:constraint=CONSTRAINT_MEMCG,nodemask=(null),cpuset=sunudock,mems_allowed=0,oom_memcg
=/sunudock,task_memcg=/sunudock,task=sh,pid=3196,uid=0
[20936.083748] Memory cgroup out of memory: Killed process 3196 (sh) total-vm:2623120kB, anon-rss:6420kB, fi
le-rss:896kB, shmem-rss:0kB, UID:0 pgtables:2140kB oom_score_adj:0
elma@DESKTOP-UEUQ2C7:~$

```

III. Conclusion

Le projet **SunuDock** a permis de concevoir un conteneur Linux opérationnel en s'appuyant exclusivement sur les mécanismes natifs du noyau, sans recourir à des outils de virtualisation tiers. Par l'implémentation rigoureuse des Namespaces (UTS, PID, Mount, Net), nous avons démontré la faisabilité d'une isolation logicielle étanche, garantissant à la fois la sécurité des processus et la conformité de l'environnement d'exécution.

L'utilisation d'appels système bas niveau, tels que clone(), chroot() et la gestion fine des Cgroups v2, a mis en évidence le fonctionnement interne des technologies modernes de conteneurisation comme Docker ou Kubernetes. La réussite des tests de stress mémoire, validée par l'intervention du OOM Killer du noyau, confirme la fiabilité de notre gestion des ressources et l'efficacité de la barrière imposée au processus fils.

En somme, ce projet constitue une immersion profonde dans l'architecture système Linux, transformant des concepts théoriques abstraits en une solution concrète, sécurisée et performante. Il valide ainsi une maîtrise solide des API système et des stratégies d'administration avancées, essentielles à la compréhension des infrastructures logicielles contemporaines.

Rapport validé par les tests de fonctionnalités et les logs du noyau.

FIN...